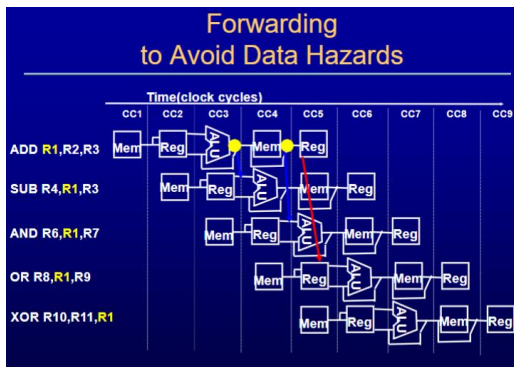


## ABSTRACT

Forward chaining is a technique to minimize the costs of data hazards in CPU pipelines without stalling execution. This report details the SystemVerilog implementation of a 5-stage MIPS datapath to execute a vector dot-product benchmark. By comparing the base dot product model against the optimized one with forward chaining, this study proves the efficiency of data bypassing. Simulation waveforms demonstrate that the forwarding unit successfully decreased instruction cycles, from 157 cycles to 103, a significant performance improvement of 54 cycles.



## 1. Introduction

The objective of this project was to design and verify a functional subset of the MIPS Instruction Set Architecture using Verilog. The microarchitecture was built as a classic 5-stage pipeline (Fetch, Decode, Execute, Memory, WriteBack) tasked with executing a specific dot-product benchmark program below.

dot\_product:

```
addu $r1 $r0 $r0 ; result = 0
```

loop:

```
beq $r7 $r0 done ; done looping?
lw $r2 0($r3) ; load a elem
lw $r4 0($r5) ; load b elem
mul $r2 $r2 $r4 ; assume this is 1 instr.
addu $r1 $r1 $r2 ; result += (*a) * (*b)
addiu $r3 $r3 #4
addiu $r5 $r5 #4
addiu $r7 $r7 #-1
j loop
```

done:

```
jr $r31
```

To ensure academic authenticity, all module and signal names were appended with the last two digits of my student ID (53). Furthermore, the input vectors for the dot product were derived from my ID. Vector A includes the 9 digits of student ID (0, 2, 0, 0, 8, 8, 6, 5, 3), and Vector B was calculated by adding the final digit (3) to Vector A using modulo-10 wrap-around arithmetic (3, 5, 3, 3, 1, 1, 9, 8, 6). The expected mathematical result of this dot product is **138**.

## 2. Baseline Architecture and Hazard Resolution

The initial implementation featured a strict baseline pipeline without any data bypassing. Because instructions frequently depend on the results of immediately preceding instructions (Data Hazards), a strict hazard\_unit\_strict53 was implemented.

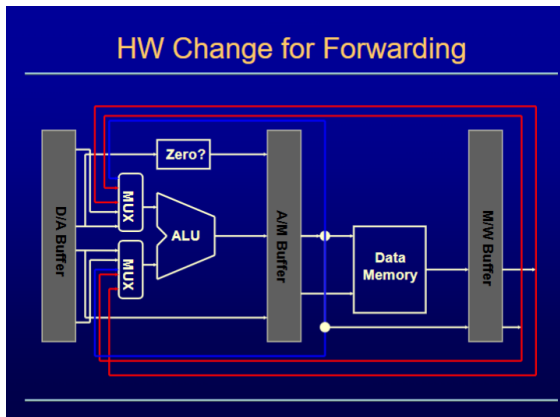
This unit continuously monitored the destination registers of the Execute, Memory, and WriteBack stages. If the Decode stage detected that a required source register was currently being written to by an older instruction in flight, the pipeline stalled. It froze the Program Counter and inserted bubbles into the control path until the required data was written back to the register file safely.

Cycle: 155 | PC: 00000028 | r1: 120 | r3: 32 | r5: 96 | r7: 1  
 Cycle: 156 | PC: 0000002c | r1: 120 | r3: 32 | r5: 96 | r7: 1  
**Cycle: 157 | PC: 00000030 | r1: 138 | r3: 32 | r5: 96 | r7: 1**  
 Cycle: 158 | PC: 00000034 | r1: 138 | r3: 36 | r5: 96 | r7: 1

## 3. Forward Chaining Optimization

To optimize the datapath and reduce execution time, a forwarding\_unit53 was implemented in the second phase of the project. Instead of stalling the pipeline for every data dependency, this unit detects when a source register in the Execute stage matches a destination register in the MEM or WB stages.

## Optimization of the MIPS Pipeline by Forward Chaining: The Dot Product Benchmark



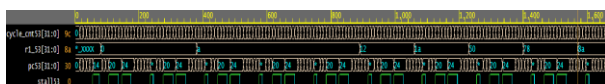
Multiplexers were added to the ALU inputs to seamlessly bypass (forward) the required data directly from the pipeline registers (EX/MEM or MEM/WB) to the ALU. With forwarding active, the strict hazard unit was replaced with a smart hazard unit (hazard\_unit\_fwd53), which only issues stalls for true Load-Use hazards (when an instruction immediately requires data currently being fetched from Data Memory) or for branch control hazards.

Cycle: 101 | PC: 00000028 | r1: 120 | r3: 32 | r5: 96 | r7: 1  
Cycle: 102 | PC: 0000002c | r1: 120 | r3: 32 | r5: 96 | r7: 1  
**Cycle: 103 | PC: 00000030 | r1: 138 | r3: 32 | r5: 96 | r7: 1**  
Cycle: 104 | PC: 00000034 | r1: 138 | r3: 36 | r5: 96 | r7: 1

### 4. Simulation Results and Performance Comparison

The design was simulated using EDA Playground (Icarus Verilog) with VCD waveform dumping to analyze cycle-by-cycle execution.

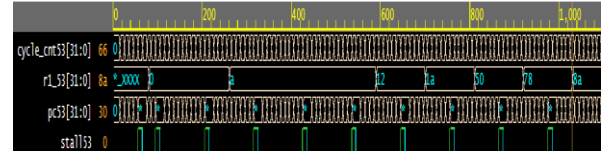
**Baseline Simulation** In the baseline architecture, the CPU successfully calculated the correct dot product of 138 (Hex 8A). However, due to the heavy insertion of pipeline bubbles required to resolve data hazards, the final result was not successfully written back to register \$r1 until **Cycle 157**.



### Optimized Simulation (With Forwarding)

Running the exact same machine code on the datapath upgraded with the forwarding\_unit53 yielded the identical

correct dot product of 138 (Hex 8A). By bypassing data directly to the ALU, the pipeline avoided almost all data-dependency stalls, allowing the final addition to write back to register \$r1 at **Cycle 103**.



### 5. Summary

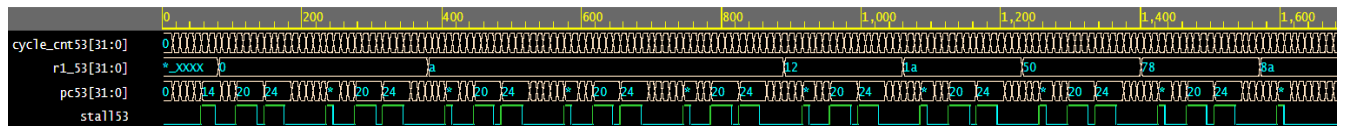
Adding the forward chaining technique to the MIPS datapath yielded a massive performance improvement. The baseline execution required 157 clock cycles to complete the dot product benchmark, while the forwarded execution required only 103 cycles. The forwarding datapath successfully **reduced total execution time by 54 cycles**, proving the critical importance of data bypassing in modern pipelined microarchitectures.

[Code Sources For Base Module](#)

[Code Sources For Forwarded Module](#)

# Optimization of the MIPS Pipeline by Forward Chaining: The Dot Product Benchmark

## Baseline Simulation



## Optimized Simulation With Forwarding

